together.

## Moving Logic into the Model

It's way outside the scope of this book to integrate with an actual payment processor, so we've cooked up a fake one named Pago, along with an implementation, which we'll see in a bit. First, this is the API it provides and a sketch of how you can use it:

```
payment_result = Pago.make_payment(
  order_id: order.id,
  payment_method: :check,
  payment_details: { routing: xxx, account: yyy }
)
```

The fake implementation does some basic validations of the parameters, prints out the payment details it received, pauses for a few seconds, and returns a structure that responds to `succeeded?`.

rails7/depot_qb/lib/pago.rb

```
require 'ostruct'
class Pago
  def self.make_payment(order_id:,
                        payment_method:,
                        payment_details:)

    case payment_method
    when :check
      Rails.logger.info "Processing check: " +
        payment_details.fetch(:routing).to_s + "/" +
        payment_details.fetch(:account).to_s
    when :credit_card
      Rails.logger.info "Processing credit_card: " +
        payment_details.fetch(:cc_num).to_s + "/" +
        payment_details.fetch(:expiration_month).to_s + "/" +
        payment_details.fetch(:expiration_year).to_s
    when :po
      Rails.logger.info "Processing purchase order: " +
        payment_details.fetch(:po_num).to_s
    else
      raise "Unknown payment_method #{payment_method}"
```